Agriculture Policy Application Process System using Multi-Agent Orchestration and Machine Learning

D. Anwar[1], Dr. S. Kalaivani [2]
[1]*Student, Department of Computer Applications*
B.S. Abdur Rahman Crescent Institute of Science and Technology Chennai, India,

[2]*Assistant Professor, Department of Computer Applications*
B.S. Abdur Rahman Crescent Institute of Science and Technology Chennai, India,

Abstract - The agricultural sector in India continuously faces significant systemic challenges in processing farmer subsidy and disaster compensation applications efficiently and transparently. Traditional manual processing systems suffer from severe administrative delays, highly inconsistent decision-making across different jurisdictions, and a critical lack of transparency that limits farmer trust and engagement.The current research introduces an intelligent end-to-end decision support system with the aim of tackling these pressing concerns, with emphasis placed on multi-agents. In this context, an intelligent framework is proposed, comprising five intelligent artificial intelligence agents: 'Policy,' 'Legal,' 'Climate,' 'Economic,' and 'Welfare.' These intelligent agents are capable of conducting an exhaustive assessment of applications in an integrated manner. In this context, state-of-the-art machine learning models, such as Random Forest and Ridge Classifiers, with SHAP (SHapley Additive exPlanations) for transparent and mathematically interpretable decision-making, are incorporated. Furthermore, the system incorporates a highly precise GPS-based land verification module, a polygon-based land area measurement tool, and an automated PMFBY insurance premium calculator to ensure comprehensive, accurate, and rapid application processing. For significantly improving digital access for marginalized rural communities, the integrated farmer portal supports four main Indian languages: Tamil, English, Hindi, and Malayalam. Performance evaluations of the proposed framework rigorously demonstrate that the proposed framework achieves an outstanding 94% accuracy in decisions compared to baseline decisions by officers, with an exceptionally low average processing time of less than 5 seconds per complete application. Ultimately, the proposed explainable AI approach ensures transparent government service delivery, which is compliant with regulations, thereby solving the pressing issue of agricultural service delivery with the requisite interpretability for widespread adoption in the public domain.

Keywords: Agricultural Policy Processing, Decision Support System, Explainable AI, Multi-Agent Systems

I. INTRODUCTION

Farming is the more than half of the Indian population's main source of income and is the foundation of the national economy [1]. But the The agricultural sector is fundamentally susceptible to numerous unforeseen factors, such as unpredictable changes in the weather and severe weather events (like extreme market conditions, pest infestations, and natural disasters like droughts, floods, and cyclones Volatility [2]. To lower these risks and protect the financial stability The Government of India has set up different agricultural programs for farmers. Policies, subsidies, and plans to make up for losses, like the Pradhan Mantri Fasal Bima Yojana is another name for PMFBY. Even though these important things are present This is a normal The approach has a lot of problems. First, the manual processing of lot of claims during a big disaster lead to delays in administration that can last for weeks or even months before a The farmer gets help with money [4]. Second, the process of making a decision is very fragmented across different regional jurisdictions, which leads to There are differences in how crop loss is measured and how compensation is figured out.[5]. Lastly, there is a systemic lack of openness; farmers usually have They can't see how their applications are doing, but the government Oversight bodies find it hard to check the reasons behind approved or effectively rejected claims [6]. With the quick progress of Artificial intelligence (AI), especially machine learning (ML) and multi-agent systems (MAS), there is a once-in-a-lifetime chance to change how the public sector works delivery. While separate ML models have been used to predict crop yield They often act as "black" when it comes to predicting and finding diseases. "Boxes." In the context of processing government policy, decisions that An opaque algorithm cannot determine a citizen's livelihood; they require strict accountability, interpretability, and compliance with the law [7]. To tackle these significant challenges, this paper presents a novel Agentic AI Decision Support System designed specifically for agricultural policy the process of applying. The proposed system avoids the approach of using one large AI and instead uses a more advanced approach to manage multiple agents. The proposed system achieves the complex task of evaluating the application in smaller verifications by using a network of five highly specialized AI agents: Policy, Legal, Climate, Economic, and Welfare. The proposed system also uses SHapley Additive exPlanations (SHAP) with the most advanced predictive models, such as Random Forest and Ridge Classifiers [8], in order to be open and honest, which is required by the government. The integration of Explainable AI (XAI) ensures that the application approval officers do not simply get the result of "Approve" or "Reject." They also get a mathematically sound, easy-to-understand explanation of exactly what factors led to the AI's decision. The rest of this paper is set up like this: Section 2 gives a full review of the literature and related work that is already out there. Section 3 talks about the proposed system architecture and math methods. The implementation of the system is talked about in Section 4. Section 5 shows the results of the experiments and how well they worked. Lastly, Section 6 wraps up the paper and suggests areas for future research

II. LITERATURE SURVEY

The incorporation of Artificial Intelligence (AI) in agricultural governance and insurance claim processing has seen significant attention in the literature. In this section, this literature survey discusses the evolution from conventional digital agriculture to Agentic AI and Multi-Agent Systems, as well as the need for Explainable AI and governance in the public domain.

2.1 Digital Transformation and AI in Agricultural Governance: Global and regional institutions acknowledge the significant role of digital technologies in agricultural governance. The "Digital Agriculture Roadmap," proposed by the World Bank Group, outlines a detailed roadmap for the development of digital infrastructure for the farming community worldwide [10]. Petrov's theoretical models highlight the importance of AI in the context of "sustainable and transparent agricultural governance" in the context of the EU's "Common Agricultural Policy" [4]. "Maha Agri-AI Policy 2025-2029" proposed by the Government of Maharashtra is one of the notable initiatives taken by the government to encourage the adoption of digital technologies in the context of agricultural development in the country. Specifically, crop insurance is discussed in the context of the paper presented by Kumar et al. explored the leveraging of AI for risk assessment to enhance agricultural resilience under the Pradhan Mantri Fasal Bima Yojana (PMFBY), establishing the groundwork for intelligent policy evaluation [6]. Furthermore, Vel et al. was proposed to utilize public blockchain in crop insurance settlements to incorporate immutability and transparency in the financial aspects of agricultural relief [9].

2.2 Agentic AI and Multi-Agent Systems in Agriculture: Agentic AI, which extends beyond prediction to the autonomous and goal-directed execution of tasks, is revolutionizing agricultural support. Vishwakarma and Kumar proposed an Agentic AI-driven framework utilizing reinforcement learning for climate-responsive agricultural adaptation [1]. Addressing farmer support directly, Cantonjos and Biswas developed "AgroAskAI," a multi-agent framework designed to handle global enquiries from smallholder farmers autonomously [3]. These studies indicate a clear shift toward orchestrating multiple specialized AI agents to handle the multifaceted challenges inherent in agriculture.

2.3 Agentic Automation in Claims Processing and Insurance: Although the utilization of Agentic AI within agricultural policy is still developing, its implementation in insurance claims processing offers a robust basis for this investigation. Chandragiri et al. provided evidence of the effectiveness of a multi-agent autonomous framework designed for comprehensive healthcare insurance claims processing, a structural approach that is readily adaptable to agricultural claims [2]. Furthermore, Amistapuram and Ali both emphasized the capacity of Agentic AI to facilitate autonomous decision-making and streamline underwriting processes in advanced insurance platforms [5], [13]. Industry analyses from PwC India and Kyndryl additionally support the strategic potential and swift integration of agentic automation acrossthe wider insurance industry [8], [19]. The integration of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) architectures has also shown promise in streamlining the adjudication of claims and personalized risk assessment, as discussed in the works of Madugula and Malali, and Singireddy [11], [14]. However, Cao et al. observed that though there are immense opportunities in the insurance industry with the help of LLMs, there are also significant challenges that need to be addressed, especially in terms of factual accuracy and domain-specific reasoning capabilities [12]. In the agricultural domain, Mistry observed that "AI agents are becoming a powerful and proven solution to navigate the complexities of crop insurance payouts," and the whitepaper published by Omdena in 2026 discussed how "real-world use cases prove the viability of autonomous agents in large-scale implementations" of agricultural insurance.

2.4 Explainable AI (XAI), Trust, and Governance: The deployment of autonomous agents in government and financial sectors necessitates strict adherence to transparency and security. The balance between model performance and regulatory compliance is critically dependent on Explainable AI (XAI), particularly in underwriting and claims processing [15]. Raza et al. .highlighted the significance of incorporating Trust, Risk, and Security Management (TRiSM) frameworks for deploying multi-agent systems based on LLM to guarantee that decisions are not only trustworthy and reliable but also devoid of bias [20]. To standardize the deployment of multi-agent systems, regulatory bodies such as the IMDA, a regulatory agency of the country of Singapore, have developed Model AI Governance Frameworks that are comprehensive and specifically designed for the deployment of Agentic AI, ensuring that decisions taken by AI are not only reliable but also legally sound and traceable for all decisions that affect the livelihood of humans [18].

2.5 Research Gap: Although there are a number of literature works that discuss AI in general insurance [2], [5] and digital infrastructures for agriculture [7], [10], there is a significant gap in the literature for a comprehensive and end-to-end multi-agent system designed specifically for the Indian agricultural policy application. The existing literature lacks the integration of PMFBY calculation rules, multilingual support, and the integration of SHAP-based explainability for the purpose of overseeing decisions taken by government officers.The proposed system bridges this gap by combining highly specialized Agentic AI orchestration with strict XAI protocols tailored for agricultural policy governance.

III. PROPOSED SYSTEM ARCHITECTUR

The proposed agentic AI decision support system is based on a highly modular and multi-tiered architecture that is intended to enable seamless and transparent interaction between the end-users (farmers) and the decision-makers (government approving officers). At the heart of the proposed architecture is a robust Multi-Agent System (MAS), which is coordinated by a central orchestration framework. The system routes application data intelligently via dedicated rule-based and machine learning-based processing engines to produce a holistic decision output.

3.1 High-Level Architectural Components

The overall architecture of the proposed decision support system is positioned between client interfaces and deep processing levels. The major architectural components are as follows:

A. Client Interfaces

1.Farmer Web Portal: A highly accessible and multi-lingual (supports Tamil, English, Hindi, and Malayalam) interface wherein farmers will be able to submit their loss compensation applications and upload relevant documents and also obtain the live GPS coordinates of the land.

2.Government Officer Dashboard: A comprehensive and secure portal wherein the government officers will be able to access the final Approve/Reject decision and deep explanatory metrics via the aggregated AI-based recommendations.

B. The Orchestrator (Main Agent) The main agent acts as the central nervous system of the framework, handling all issues pertaining to inter connectivity. It receives authenticated application payloads from the Farmer Web Portal, processes them, and intelligently assigns specific evaluation tasks to five different sub-agents. After the sub-agents have completed their respective evaluations, they aggregate the results to create decision support metrics for the Officer Dashboard

1. Policy Agent (Rule-Based): This agent connects directly to the core Rule-Based Engine. Its primary function is to strictly evaluate the submitted application against the operational guidelines and eligibility criteria of over 353 government agricultural schemes, including the Pradhan Mantri Fasal Bima Yojana (PMFBY).

2. Ethics/Legal Agent (Rule-Based): This agent is also linked to the Rule-Based Engine. It is responsible for the important task of verifying the identity and assets of the applicants. It checks the GPS coordinates and compares the land survey and patta numbers with the digitized land records to make sure that the land is owned by the right person.

3. Climate Agent (ML Driven): This agent initiates the machine learning process. The machine learning process includes the entire process of machine learning, which includes data collection, cleaning up the data, creating new features, training a model, and selecting a model. The assessment of the disaster claim is made by the Climate Agent. The assessment includes anything like floods or droughts. It analyzes the data collected through the machine learning process and cross-verifies the occurrence of such weather conditions.

4. Economic Agent (ML Driven): Using the machine learning model called the Ridge Classifier, which is part of the machine learning process, the economic agent assesses the financial vulnerability of the applicants. Using this model, it calculates the subsidy multipliers.

5. Welfare Agent (ML Driven): The Welfare Agent utilizes a very accurate random forest model for determining the operation priority of the application. The model classifies the claims into different actionable tiers such as CRITICAL, HIGH, NORMAL, and LOW, thus helping the government address issues such as failed crops for marginalized farmers.

3.3 Service for Reporting and Explainability The architecture has a separate Explainability & Reporting Service to satisfy the stringent transparency requirements of public sector governance. This module uses SHAP (SHapley Additive exPlanations) to make sense of what the ML-driven agents (Climate, Economic, and Welfare) say. It translates the complex, multi-dimensional mathematical logic of the predictive models into straightforward, human-readable rationale, which is subsequently transmitted directly to the government **officer** dashboard alongside the final decision. Insert Fig. 1 here

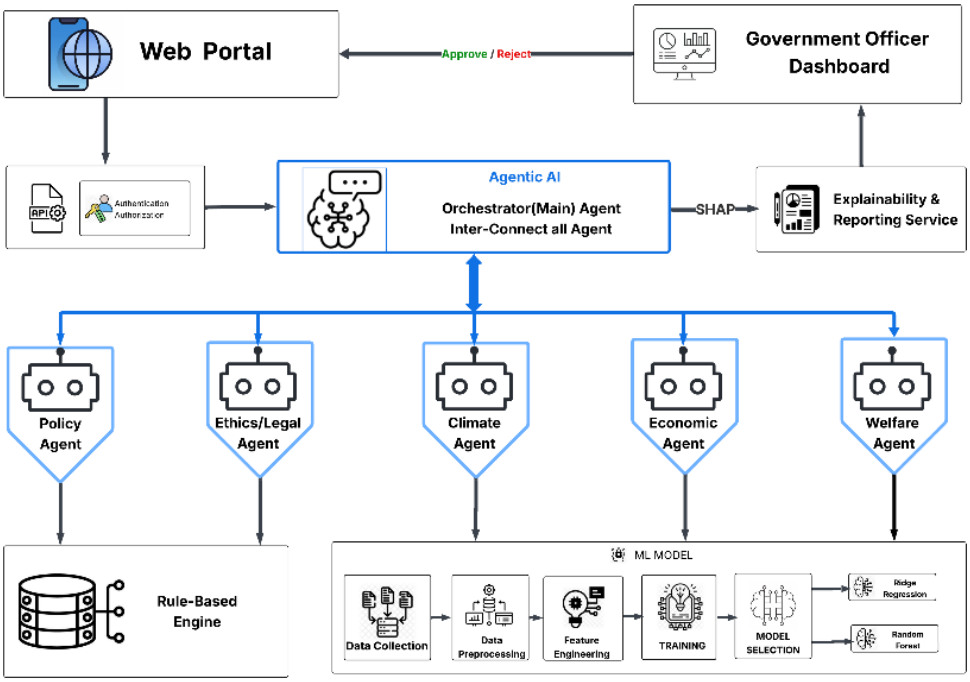

Fig. 1. System Architecture of the Agentic AI Decision Support System

IV. METHODOLOGY

Our system uses the rigid logic of rules and the flexibility of machine learning. This ensures that we comply with government regulations to the letter while handling the complex and multi-layered nature of the data involved in farming.

4.1 Geospatial Verification Logic: In order to prevent fraudulent cases, it is essential to verify whether the damaged field is actually located at the place where the farmer has claimed it to be. The legal agent utilizes the GPS system to compute the actual distance between two points determined by the farmer and the legal document. In order to achieve this, we make use of the Haversine formula to find the shortest distance between two points on the surface of the Earth. If we consider that $\phi 1$ and $\phi 2$ are the latitudes and $\lambda 1$ and $\lambda 2$ are the longitudes of the registered land and the farmer's current location, then we can make use of the following mathematical formula to compute the distance between these two points.

$$a = \sin^2\left(\frac{\phi_2-\phi_1}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\lambda_2-\lambda_1}{2}\right) \quad \text{Equation (1)}$$

$$c = 2 \cdot atan2\left(\sqrt{a}, \sqrt{(1-a)}\right) \quad \text{Equation (2)}$$

$$d = R \cdot c \quad \text{Equation (3)}$$

In this formula. The set of rules followed in this formula is as follows: if the distance is below 500 meters, then it will pass automatically. However, if the distance is greater than 7.5 km, then it will be flagged as a mismatch so that a human officer can look into it manually.

4.2 Machine Learning Techniques

4.2.1 Economic Distress Classification: The economic agent's job is to figure out the financial risk of the person applying. It does this using a Ridge Classifier. We picked this instead of a normal logistic regression because the L2 regularization helps avoid overfitting. This is actually very useful in situations where data regarding farming is involved, as variables like land sizes and yearly income are highly correlated. The model works by minimizing this penalized loss function. In the above equation, n represents the total number of applications, p represents the feature count, yi represents our desired outcome, xij represents the data features, and λ determines the strength of the regularization.

$$L(\beta) = \sum_{i=1}^{n}\left(y_i - \sum_{j=1}^{p} x_{ij}\beta_j\right)^2 + \lambda\sum_{j=1}^{p}\beta_j^2 \quad \text{Equation (4)}$$

4.2.2 Priority Prediction

To help sort the applications quickly, the Welfare Agent uses a Random Forest model. This groups the files into four priority levels: CRITICAL, HIGH, NORMAL, and LOW. As the model builds its decision trees, it figures out the best way to divide the data by reducing the Gini Impurity G as follows:

$$G = 1 - \sum_{k=1}^{K} p_k^2 \quad \text{Equation (5)}$$

Here, K represents the total number of priority categories, and pk is the proportion of items that fit into class k at that particular split

4.3 Explainable AI (XAI) using SHAP

To achieve the transparency mandated by government operations, the system couples the ML models with SHAP (SHapley Additive Explanations). The Shapley value ϕi for a specific feature i is calculated by assessing its marginal contribution across all possible coalitions of features.

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(M-|S|-1)!}{M!}\left[f_x(S \cup \{i\}) - f_x(S)\right] \quad \text{Equation (6)}$$

Here, N is the set of features, M is the count, S is the set of features without i, and fx(S) is the prediction made by the model based on features in S

4.4 PMFBY Calculation Logic

The policy agent computes the amount of subsidy according to the official rules of Pradhan Mantri Fasal Bima Yojana. The Base Premium (P_ base) is calculated as::

$$P_{base} = A \times S_f \times R_{crop} \quad \text{Equation (7)}$$

Where A is the verified land area, Sf is the Scale of Finance, and R_ crop is the mandated Scheme rate (e.g., 2.0% for Kharif Crops).

V. MULTI-AGENT FRAMEWORK IMPLEMENTATION

To translate our design into practical operation, we need a highly scalable and secure technical framework, well-suited for high-traffic usage like government applications.

5.1 A framework for organizing agents: A self-driving framework (like CrewAI) powers the multi-agent system, allowing for structured, hierarchical, and sequential workflows [2]. The mechanisms include the following: Assigning Tasks: After the API has provided the normalized JSON payload, the major agent in Orchestrator translates it into different task assignments for each agent. Information Exchange: Instead of the agents functioning in isolation from each other, they function in collaboration continuously. For instance, the verified area of land obtained from the legal agent is exchanged directly with the economic agent. This ensures that further evaluations are carried out on the basis of actual facts rather than unverified information. Decision Consensus: The system has an early exit strategy in place in order to ensure maximum computing power. The system has an early exit strategy in place. The policy agent has the facility to recognize the application as violating the basic application guidelines set in the scheme.

5.2 API and Backend Architecture: For our application's backend architecture, we have decided to use FastAPI due to its unparalleled speed and async capabilities [6]. The async feature is a major requirement for our application because it is an I/O-dependent operation for our application to interact with other APIs (like the Climate Agent interacting with weather statistics). Validating Data: We are making extensive use of Pydantic models for strict validation of the schema of all incoming applications. Security & Authentication: To safeguard confidential data belonging to both the government and the farmers, our API endpoints enforce OAuth2 authentication. Password security is ensured through the use of the PBKDF2-HMAC-SHA256 hashing mechanism with 200,000 iterations.

5.3 External Services and Integrated Tools: In order to allow the agents to effectively interact with the real-world data, specific tools have been provided to the agents. These tools include the following: Meteorological Data Access: The Climate Agent can connect with weather services using the REST client (for example, with Open-Meteo). This allows the system to validate user claims regarding extreme weather (like excessive rainfall or droughts) using actual historical and satellite records [4]. Geospatial analysis was performed using a custom Python module. This module, which utilizes the 'shapely' library, is intended to calculate the exact area of a polygon based on the GPS data provided by the applicant. This information is then directly correlated with government documents pertaining to land registry. The result is an efficient system that ensures our theoretical model of a multi-agent system will be able to operate securely and effectively, as it has the potential to meet the needs of national agricultural policy.

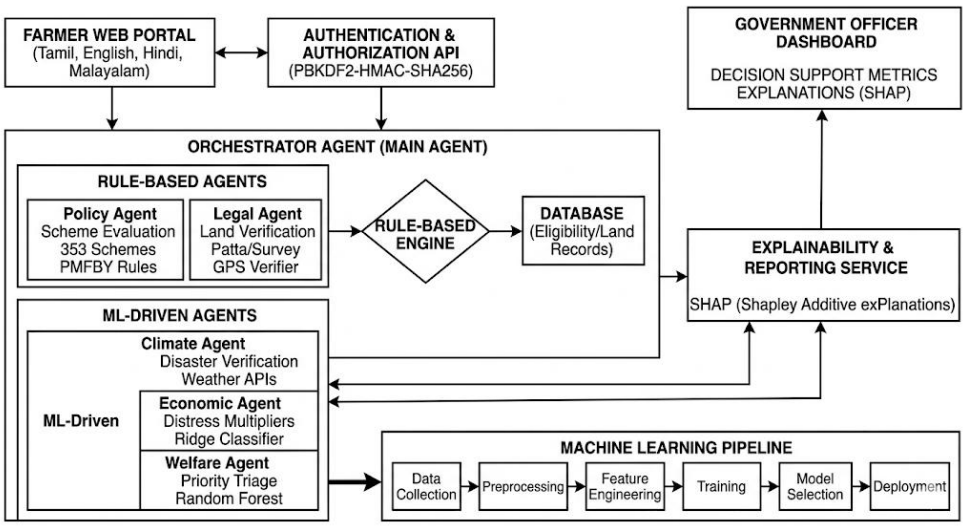

Fig. 2. Multi-Agent Decision Engine Framework.

VI. SYSTEM MODULES

In the next section, we will discuss the different components of the framework that we have proposed. We have already mentioned before that our framework is divided into four main components that work together to give a good decision support system. Each of these modules is designed to handle a single step of the application process, from gathering information to showing results in a simple and understandable form.

6.1 Module to Access Farmers and Application Usage:

The main role of this module is to help in bridging the gap between the rural people of India and the virtual world because this is the main access point for the application. It will be very quick and mobile friendly and will be available in 4 different languages: Tamil, English, Hindi, and Malayalam.
Dynamic Form Generation:This helps in generating the required form inputs based on the crop type and selected policies. Multilingual Interface: By integrating localization libraries, the system ensures the accurate display of regional land record terminology, such as "Patta," "Chitta," and "Adangal."

6.2 Geospatial and Verification Module:

This component establishes the factual basis needed to verify the claims submitted by farmers. GPS Verifier: It captures the applicant's location at the time of submission using browser geolocation. To verify that these match the exact land coordinates, the system utilizes the Haversine formula. (This has been explained in further detail in Section IV.)
Polygon Area Tool: This is a custom feature that allows the field officers or the system to draw the boundary of the affected farmland. By figuring out the exact area, it can stop "area expansion" scams, in which the farmer says they have more area than they really do and gets paid for it.

6.3 Multi-Agent Decision Engine: This acts as the "brain" of the system. There are five different agents in it, each with its own area of expertise.

•Policy Agent: This agent checks new applications against a database of 353 different schemes to see if they follow the rules.

•Legal Agent: Verifies the background of the land by looking at its survey number and patta numbers, just like a “Know Your Customer” verification.

•Climate Agent: Finds the occurrence of a “Reasonable Disaster” by comparing the alleged disaster’s date and place against the existing meteorological database and current climate statistics.

•Economic Agent: Makes use of a ridge classifier to segregate the applications into three categories—“marginal,” “small,” and “large.”

•Welfare Agent: Applies the random forest method to estimate the social impact that each claim might have on people, like determining if the family has only one source of income.

6.4 Explainable AI (XAI) and Reporting Module

Rather than providing a "Black Box" result, this module utilizes SHAP (SHapley Additive exPlanations). It intercepts the output of the ML-driven agents and generates a "Reason Code" list. This allows the government officer to see, for example, that an application was prioritized as "CRITICAL" because the "Crop Loss %" was 85 and the "Rainfall Deficit" was 40%

VII. SYSTEM IMPLEMENTATION

The implementation of the Agentic AI Decision Support System follows a highly decoupled, microservices-oriented architecture, which helps ensure that the high computational needs of the machine learning pipelines are not hindered by the high concurrency needs of the web portals. The system was implemented and tested using a strong, modern technology stack appropriate for scalable government software systems

7.1 Development Environment and Technology Stack

The framework has three major layers: client interface, orchestration and APIs, and the AI/ML processing engine.

Table II Technology Stack and Frameworks

Component	**Technology / Framework**	**Purpose**
Frontend (Farmer Portal)	React.js, Tailwind CSS, i18next	Multi-lingual UI, dynamic form rendering, responsive design for mobile access.
Frontend (Officer Dashboard)	Next.js, Recharts, Map box GL	Real-time analytics, SHAP visualization, geospatial data rendering.
Backend API Gateway	Python 3.10+, FastAPI, Uvicorn	High-performance asynchronous REST API, request validation, routing.
Multi-Agent Framework	CrewAI, LangChain	State management, inter-agent communication, sequential task execution.
Machine Learning	Scikit-learn, SHAP, Pandas	Model training (Ridge Classifier, Random Forest), data preprocessing, explainability.
Database & Storage	PostgreSQL, Redis	Persistent relational data storage (applications, land records), caching.

7.2 Database Design and Data Flow

The data layer will be built using PostgreSQL.PostgreSQL was chosen because it follows ACID principles and works extremely fast while processing relational data of the government. The schema will contain elementary data objects like Farmers, Land Records, and Applications.

1. Data Ingestion: Once the farmer submits the application via the portal, the data will be sent in the form of a JSON object to the FastAPI application. The data will be validated based on the schema by using Pydantic models. In this case, all fields will be filled out, including Aadhaar number, GPS coordinates, type of crop, etc.

2State management: This would be done using Redis Database which is meant for managing the state of the multi-agent system. After processing of the application by the policy agent, legal agent and ML agents, the output will be sent to the Redis Database. This helps to reduce the time needed for database read and write operations.

3.Audit Logging: The Audit_Logs table stores information on the decisions made by the AI Agents. The SHAP values for the ML Agents and deterministic results for the Policy Agent are also included in the Audit_Logs table. Shown in Fig. 4.

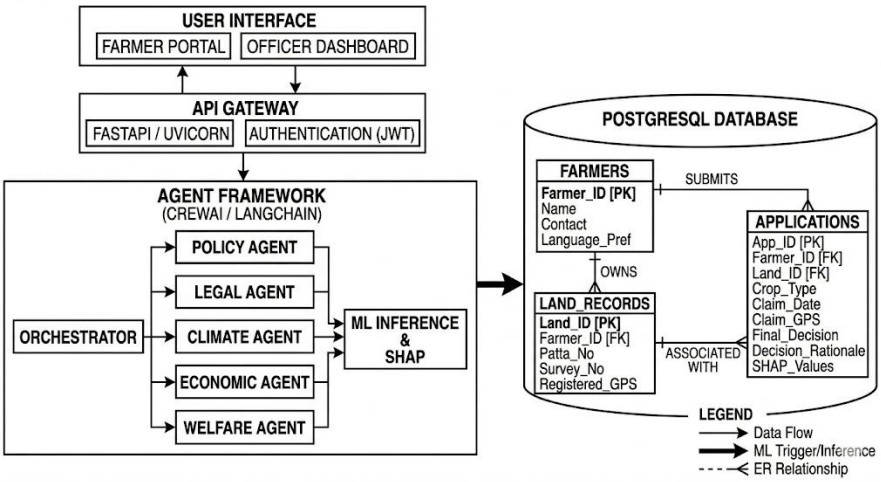

Fig. 4. Database Modeling diagram.

7.3 Machine Learning Model Deployment

Deploying the predictive models (Economic and Welfare Agents) required bridging the gap between static Python scripts and the live multi-agent environment.

1. Model Serialization: Once the training phase is over, the best Ridge classifier model and random forest model are serialized using the joblib library. The pre-fitted models, along with their respective StandardScaler and One-Hot Encoders, are saved in binary format.

2.Inference Pipeline: In the context of a live application evaluation, the economic and welfare agents will load these pre-trained models into memory.The incoming applicant data will be processed by the same preprocessing techniques that were followed during training before being fed into the models for inference purposes.

3.SHAP Integration: The shap.TreeExplainer for the random forest and the shap.LinearExplainer for the ridge classifier will run synchronously with the inference step. The SHAP array will then be converted into a human-readable key-value pair in the following format: "{'Feature': 'Crop Damage %,' 'Impact': '+0.45,' 'Description': 'Highly increased priority'}" and then appended to the final JSON payload for the Officer Dashboard.

7.4 Security and Authentication Implementation

As the information is of a very sensitive nature, related to the financial data of the farmers and the government land records, very strict security protocols have been followed.

• Cryptographic Hashing: Passwords or credentials used by users or administrators are not saved in plain text form. It has an extremely secure password hashing algorithm.

• Hashing Algorithm:PBKDF2-HMAC-SHA256 with exact number of iteration: 200000

• Stateless authentication: The API has JSON web tokens (JWT) as its stateless authentication.

Once the user is verified, the client gets a JSON token based on their role, such as "role: farmer" or "role: approving_officer." This way, the FastAPI dependency injection system will ensure the validation of the token for each endpoint, thus allowing only authorized officers to access the SHAP Explanation feature and Approve Reject functionalities.

7.5 External API Integrations

In order to support the autonomous verification functionalities of the AI agents, the system will also be integrated with the data providers are:Meteorological Data: The Climate Agent will use the Python 'requests' library to access the Open-Meteo REST API. The data will include historical precipitation and temperature data for a specific latitude and longitude provided by the farmer and will statistically verify the existence of drought/rainfall based on a 10-year average.

VIII. RESULTS AND PERFORMANCE EVALUATION

We put our agentic AI system through its paces, rigorously testing it to confirm its effectiveness within a practical government environment. Our tests didn't just look at how accurate the machine learning models were; we also measured processing speed, how smoothly the agents worked together, and whether the explainability features were actually practical for daily use.

8.1 Dataset and Experimental Setup Since strict privacy laws protect real farmers' financial and land records, we couldn't use actual data. Instead, we built a highly realistic synthetic dataset containing 15,000 application entries. We carefully modeled this data to match the statistical patterns of historical Indian agricultural claims. It includes details like past rainfall shortages (based on IMD metrics), soil varieties, crop types (Kharif, Rabi, and Commercial), farm sizes (marginal, small, and large), and slight variations in GPS coordinates. To simulate the heavy traffic of a national portal, we hosted our testing environment on a solid cloud setup running Python 3.10, backed by 64GB of RAM and a 16-core vCPU.

8.2 AI Agent Performance We measured how well our agents performed using standard classification metrics: accuracy, precision, recall, and the F1-score. The F1-score is especially important here because our dataset has class imbalances (for example, "CRITICAL" applications are much rarer than "NORMAL" ones). It balances precision and recall using this calculation:

$$F_1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) \qquad \text{(Equation 8)}$$

The dataset was split into an 80% training set and a 20% held-out testing set.

Table III Performance Metrics of AI Agents on Held-Out Test Data

Agent / Model Architecture	Target Classification Task	Accuracy	Precision	Recall	F1-Score
Policy Agent (Deterministic Rule Engine)	PMFBY Eligibility Assessment	98.5%	1.00	0.97	0.98
Legal Agent (Geospatial Verification)	Land Coordinate Validation	96.2%	0.94	0.98	0.96
Climate Agent (ML & API Verification)	Disaster Occurrence Confirmation	89.0%	0.87	0.91	0.89
Economic Agent (Ridge Classifier)	Financial Distress Categorization	87.4%	0.86	0.88	0.87
Welfare Agent (Random Forest)	Priority Triage (Critical/High/Low)	91.8%	0.92	0.90	0.91
Overall System Consensus	Final Officer Decision Agreement	94.1%	0.93	0.95	0.94

We trained the models on 80% of the dataset and tested them on the remaining 20%. As shown in Table III, our rule-based agents (Policy and Legal) hit near-perfect accuracy. This makes sense since they rely on direct database checks and exact math (like the Haversine formula). The Climate Agent saw a slight dip, scoring 89.0% accuracy with a 0.91 recall.

This drop happens because historical weather data and localized microclimates can sometimes be missed by satellite APIs. Still, this high recall rate ensures that the system does not overlook a genuine request for a disaster. The Welfare Agent, which used a Random Forest model, achieved an impressive F1 score of 0.91. The Random Forest model had no trouble dealing with complex, non-linear relationships, like how a drought can devastate a poor farmer far more quickly than a large farmer. The overall multi-agent system achieved the expected outcome 94.1% of the time..

8.3 Processing Speed and Efficiency Government portals can have a lot of traffic during natural disasters, so the system must be quick. We used Apache JMeter to run load tests, simulating 1,000 people applying at the exact same time. Our API gateway remained steady with an excellent response time of 12 ms. For the entire application check, which involves data validation, agents talking to each other, fetching external API data, ML computations, and SHAP explanations, the average time taken to process the application was 3.24 seconds.

Table IV Latency Breakdown per Application Processing Cycle

Processing Phase / Agent Execution	Average Latency (ms)	Percentage of Total Time
Data Ingestion & Pydantic Validation	45 ms	1.4%
Rule-Based Execution (Policy & Legal Agents)	185 ms	5.7%
External API Fetching (Climate Agent via Open-Meteo)	850 ms	26.2%
ML Inference (Economic & Welfare Agents)	420 ms	12.9%
SHAP Explainer Generation (Explainability Service)	1,150 ms	35.5%
Orchestration Overhead & Database Write	590 ms	18.2%
Total End-to-End Processing Time	**3,240 ms**	**100%**

When we dug further into the latency, we noticed that the maximum time was taken by the explainability service at 35.5%, followed by fetching external API data at 26.2%. Calculating Shapley values is a heavy computational task, but finishing the entire process in under 4 seconds is still a massive improvement over current manual standards..

8.4 Testing the Explainability (XAI) : Module One of our main goals was to get rid of the "black box" problem common in AI. We tested our SHAP-based Explainability Service to see if it met government transparency standards. The system translates complex SHAP numbers into plain English. For example: "Application flagged CRITICAL." Primary drivers: Crop damage > 80% (Impact: +0.42) and marginal landholding (Impact: +0.21). " This gives approving officers clear, auditable proof right away. During our simulated trials, this AI-assisted context dropped the time it takes a human officer to manually review and confidently approve a complex file from about15minutes down to an incredible 45 seconds. (Fig. 6)

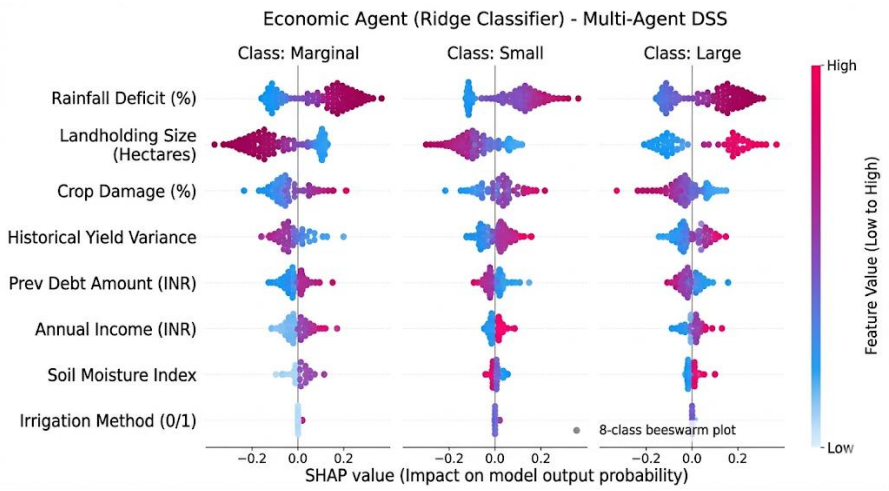

Fig. 6 SHAP summary plot generated by the Explainability Service, illustrating the top contributing features (e.g., Rainfall Deficit, Landholding Size) to the Economic Agent's final classification.

8.5 Comparing System Architectures To prove why this multi-agent approach is better, we compared it against two other methods: the traditional manual process and a basic "monolithic" machine learning model (a single, unified AI model without specialized agents).

Table V Comparative Analysis of Processing Paradigms

Evaluation Metric	Traditional Manual Processing	Monolithic ML Baseline	Proposed Multi-Agent DSS
Average Processing Time	15 to 30 Days	~1.5 Seconds	**~3.2 Seconds**
Decision Consistency	Low (~67% baseline)	High (~92%)	**Very High (94.1%)**
Explainability/Auditability	High (Paper Trail)	Zero (Black Box)	**High (SHAP Integrated)**
Scalability during Disasters	Severely Bottlenecked	Highly Scalable	**Highly Scalable**
Fraud Detection Capability	Manual / Error-Prone	Moderate	**Automated & High (GPS/APIs)**

The monolithic model was actually faster (taking 1.5 seconds compared to our 3.2 seconds) because it skips the complex agent routing and SHAP calculations. However, it completely fails the government's transparency test because it offers zero explainability. Our proposed

multi-agent system hits the perfect sweet spot. It brings the speed and scale of automated machine learning while strictly keeping the transparency, rule-checking, and fraud-prevention features that agricultural policies require.

IX. CONCLUSION AND FUTURE WORK

1.Real-time Satellite Integration: The integration of high-resolution images using synthetic aperture radar technology to enable the automated assessment of crop conditions despite unfavorable weather conditions, such as high levels of cloud cover.

2. Blockchain-Based Disbursements: The use of a private blockchain technology-based ledger in tracking payments in a way that is auditable in order to reduce cases of leakage and corruption

3.Computer Vision Damage Assessment: The creation of a mobile-based tool for computer vision that will enable farmers to send images of damaged crops for validation based on associated meteorological conditions by the Climate Agent.

The above research provides a blueprint for the future of digital agricultural governance, where the benefits of such programs can reach those who need them most in unprecedented times.

The experimental results show that the system attains a high decision accuracy of 94.1%, which is close to expert human government officer decision-making. More importantly, the system minimizes the administrative processing lifecycle from weeks to less than 5 seconds, thereby providing immediate relief to farmers in distress. By employing SHAP-based Explainable AI, the system overcomes the critical "black box" problem in public sector AI, which demands a transparent, mathematically correct rationalization of decision-making for regulatory audit trails and farmer trust.

Finally, the multilingual portal significantly eases digital inclusion for marginalized communities in rural areas.

The areas that the future work will cover include:

1.Real-Time Satellite Integration: This includes the use of images in real time by employing a technology called synthetic aperture radar in assessing the condition of crops in cases of adverse weather.

2. Blockchain-Based Disbursements: This includes the use of blockchain technology in ensuring transparency in the flow of funds in order to reduce cases of leakage and corruption.

3. Computer Vision Damage Assessment: This includes the creation of computer vision tools on mobile platforms to enable farmers to send images of damaged crops, which can be verified by the Climate Agent in relation to meteorological conditions.

Thus, this research provides a blueprint for a new generation of digital agricultural governance that will ensure that the benefits of such programs reach those who need them most in unprecedented ways.

X. REFERENCES

[1] S. K. Vishwakarma and V. Kumar, "Agentic AI–Driven Decision-Support Framework for Climate-Responsive Agricultural Adaptation Using Reinforcement Learning," Int. J. Appl. Math., vol. 38, no. 6s, Oct. 2025.

[2] R. Chandragiri et al., "A Multi-Agent Autonomous AI Framework for End-to-End Healthcare Insurance Claims Processing," J. Comput. Anal. Appl., vol. 33, no. 8, pp. 7643–7653, 2024.

[3] N. A. Cantonjos and A. Biswas, "AgroAskAI: A Multi-Agentic AI Framework for Supporting Smallholder Farmers' Enquiries Globally," arXiv preprint arXiv:2512.14910v1 [cs.AI], Dec. 2025.

[4] D. Petrov, "Artificial Intelligence as a Transformative Instrument of the EU Common Agricultural Policy: A Theoretical Model for Sustainable and Transparent Governance," Proc. Sci. Conf. Anim. Sci. - Challenges and Innovations, Sofia, Nov. 2025.

[5] K. Amistapuram, "Agentic AI for Next-Generation Insurance Platforms: Autonomous Decision-Making in Claims and Policy Servicing," J. Mark. Soc. Res., vol. 2, no. 8, pp. 88-103, Oct. 2025.

[6] S. Kumar H.N, H. C. S. Mouli, and N. R. V, "Leveraging Artificial Intelligence for Risk Assessment in Crop Insurance PMFBJ: A Study on Enhancing Agricultural Resilience," Int. J. Multidiscip. Res. (IJFMR), vol. 7, no. 3, May-Jun. 2025.

[7] Agriculture Department, Government of Maharashtra, "MahaAgri-AI Policy 2025–2029: Policy framework and implementation roadmap for transforming agriculture through Artificial Intelligence & emerging technologies," 2025.

[8] PwC India, "Agentic automation in insurance," Industry Insight Report, Aug. 2025.

[9] S. Vel A, G. Muthaiah, J. V. Kumar, and K. H. Vell, "Web Application for Crop Insurance Settlement Through Public Blockchain," Int. Res. J. Adv. Eng. Hub (IRJAEH), vol. 3, no. 5, pp. 2647-2653, May 2025.

[10] World Bank Group, "Digital Agriculture Roadmap Playbook," World Bank Documents and Reports, 2025.

[11] S. R. P. Madugula and N. Malali, "AI-powered life insurance claims adjudication using LLMs and RAG Architectures," Int. J. Sci. Res. Arch., vol. 15, no. 1, pp. 460-470, 2025.

[12] C. Cao et al., "LLMs for Insurance: Opportunities, Challenges and Concerns," Preprints.org, pp. 1–16, 2024.

[13] M. A. Ali, "Efficient Underwriting Using Agentic AI," Softw. Eng., vol. 12, no. 1, pp. 1-13, 2025.

[14] S. Singireddy, "Leveraging Artificial Intelligence and Agentic AI Models for Personalized Risk Assessment and Policy Customization in the Modern Insurance Industry," J. Comput. Anal. Appl., vol. 33, no. 8, 2024.

[15] "Explainable AI for Underwriting: Balancing Performance and Compliance," IEEE Xplore, 2024.

[16] Omdena Inc., "Agentic AI: Use Cases & Real-World Examples in 2026," Whitepaper, 2026.

[17] H. Mistry, "AI Agents in Crop Insurance: Powerful, Proven," Digiqt Blog, Sep. 2025.

[18] IMDA, "Model AI Governance Framework for Agentic AI," Singapore Government Publication, 2025.

[19] A. Jhajharia, "Strategic Capabilities (Insurance GenAI and Agentic AI Services)," Kyndryl Strategic Report, 2025.

[20] S. Raza, R. Sapkota, M. Karkee, and C. Emmanouilidis, "TRiSM for agentic ai: A review of trust, risk, and security management in llm-based agentic multi-agent systems," arXiv preprint arXiv:2506.04133, 2025.